Theory Behind Splink: The Fellegi-Sunter Model and EM, Estimation in Splink, and Extensions

Cooper Desmond

November 10, 2026

1 Overview

This document explains how Splink estimates and applies the Fellegi-Sunter model of probabilistic record linkage. Fellegi and Sunter (1969) showed that the optimal rule for classifying a pair of records as a link or a non-link depends on the ratio between the probability of the observed agreement pattern among matching pairs and its probability among non-matching pairs. Splink (Linacre et al., 2022) is an open-source Python package that implements this model in SQL so that it can run on large datasets.

Sections 2 through 7 derive the model. They set out the notation, the conditional independence assumption, the posterior match probability and the expectation-maximization (EM) updates for the parameters. Section 8 describes how Splink departs from textbook EM. It estimates the prior from deterministic rules and the non-match parameters from random sampling, runs EM within blocks and adjusts for term frequency. The section ends with prediction. Section 9 describes three extensions I made to the Splink workflow, which are a sampling scheme for EM training when individuals have several ranked observations, parallel prediction, and one-to-one comparison enforcement. Appendix A proves that an EM update cannot decrease the likelihood. This paper considers Splink v4, so newer versions may differ in implementation.

Contents

1 Overview **1**

2 Setup and notation **2**

3 Parameters **2**

4 Weight bearing assumption **3**

5 Posterior **3**

6 EM theory **4**

7 EM steps **5**

8 Splink workflow **7**
8.1 Prior from deterministic rules 8
8.2 u from random sampling 8
8.3 m from blocked EM 9
8.4 Term frequency adjustments 10
8.5 Prediction 12

9 Extensions **13**
9.1 Sampling with multiple ranked observations 13
9.2 Parallel prediction 16
9.3 One-to-one comparisons 16

Appendix A. Maxing Q never decreases ℓ **18**

References **20**

2 Setup and notation

The goal of training a Fellegi-Sunter (FS) model is to estimate $P(M_i = 1|\text{what we observe about pair i})$ where for any given pair of records i we have the unobserved indicator:

$$M_i = \begin{cases} 1 & \text{if the pair is a match} \\ 0 & \text{if the pair is not a match} \end{cases}$$

The "what we observe about pair i" is collapsed into a space defined by F features where each feature has a set of levels that describes different levels of agreement between the pair on that feature. The features and the level values they take are described by the comparison vector $\gamma_i = (\gamma_{i1}, \ldots, \gamma_{iF})$ where γ_{ik} represents the level of feature k for pair i. Each feature has a null level as the top of the level waterfall, then the possible agreement levels are ordered from strongest to weakest. For dob, the levels look like (j = 1 for either null, j = 2 for exact match, j = 3 for DL <= 1). So if dob is the first feature and pair 2 has an exact match then the comparison vector would be $\gamma_2 = (2, \ldots)$. Within the model, the comparison vector represents what we observe about each pair.

The goal is to 1) use the comparison vectors to train the model parameters unsupervised and 2) output a prediction probability given the comparison vectors.

3 Parameters

The three parameters the model needs to train are

$$\theta = (\lambda = P(M_i = 1),\ m_{kj} = P(\gamma_{ik} = j|M_i = 1),\ u_{kj} = P(\gamma_{ik} = j|M_i = 0))$$

for given pair i, feature k, and feature level j.

Splink handles the null level separately. It gives the null level no m or u, so a null contributes a factor of one to the products in the sections below. This amounts to assuming that whether a feature is null does not depend on match status. The m_{kj} and u_{kj} of the other levels are then probabilities among the pairs that are non-null on feature k.

4 Weight bearing assumption

The main FS assumption is that, given the match status, the resulting levels of each feature are independent of one another so we can write:

$$P(\gamma_i \mid M_i = 1) = \prod_{k=1}^{F} m_{k\gamma_{ik}}, \qquad P(\gamma_i \mid M_i = 0) = \prod_{k=1}^{F} u_{k\gamma_{ik}}$$

This makes the rest of the model tractable. It is also frequently violated in practice. For example, among non-matches, agreement on a gender feature is near certain given agreement on first name, agreement on last name is near certain given agreement on household, even agreement on first name given agreement on last name is more common than implied by the parameters because names cluster. In Splink, this is handled by combining highly conditionally dependent features. Say combining address and phone number, which both measure household evidence, into a singular household feature that uses both variables in its levels. However this cannot solve all the issues. The Splink maintainers acknowledge this and note that when you suspect that the conditional independence is bad (like when adding address into the model, because it is conditionally dependent on last name and other household variables), you should treat the probabilities as simply ordinal scores, and that in their experience those ordinal scores do a good job at ranking matches.

Xu et al. (2019) study how much these violations matter for matching. They note that ignoring conditional dependence biases the parameter estimates, but argue that a linkage model should be judged by how accurately it classifies pairs. Using three real linkage datasets and a simulation study, they find that the FS model is generally robust to violations of conditional independence and matches about as accurately as more complex models that allow for dependence. They give two caveats. First, the FS model does worse when the match prevalence is close to 0% or 100% and the dependence is in the dominating class, although this matters less when highly discriminating features are used. Second, a model with the wrong dependence structure can match less accurately than the FS model, so they caution against using such models blindly.

5 Posterior

By the chain rule we know that:

$$\begin{aligned} P(\gamma_i, M_i = 1) &= P(M_i = 1)\, P(\gamma_i \mid M_i = 1) = \lambda\, P(\gamma_i \mid M_i = 1) \\ P(\gamma_i, M_i = 0) &= P(M_i = 0)\, P(\gamma_i \mid M_i = 0) = (1 - \lambda)\, P(\gamma_i \mid M_i = 0) \end{aligned}$$

So we can marginalize out M_i such that:

$$P(\gamma_i) = \sum_{m \in \{0,1\}} P(M_i = m)\, P(\gamma_i \mid M_i = m) = \lambda\, P(\gamma_i \mid M_i = 1) + (1 - \lambda)\, P(\gamma_i \mid M_i = 0)$$

Then here invoke the conditional independence assumption above to get:

$$P(\gamma_i \mid \theta) = \lambda \prod_k m_{k,\gamma_{ik}} + (1 - \lambda) \prod_k u_{k,\gamma_{ik}}$$

Finally, we can get the posterior with Bayes' rule:

$$p_i(\theta) = P(M_i = 1 | \gamma_i, \theta) = \frac{P(M_i = 1|\theta) P(\gamma_i | M_i = 1, \theta)}{P(\gamma_i | \theta)} = \frac{\lambda \prod_k m_{k,\gamma_{ik}}}{\lambda \prod_k m_{k,\gamma_{ik}} + (1 - \lambda) \prod_k u_{k,\gamma_{ik}}}$$

Turning it into the odds form in log space, we get Splink's "match weight" given the odds form of lambda $\pi = \frac{\lambda}{1-\lambda}$ and the Bayes factor $K_{kj} = m_{kj}/u_{kj}$:

$$\frac{p_i}{1-p_i} = \pi \prod_k K_{k\gamma_{ik}}, \quad \log_2 \frac{p_i}{1-p_i} = \log_2 \pi + \sum_k \log_2 K_{k\gamma_{ik}}$$

The $\log_2 \pi$ is the prior and each feature contributes the "bits" from the observed level, where each bit in log space doubles the odds. This constructs how the evidence provides the odds, but it is conditioned on knowing θ which is the subject of the next section.

6 EM theory

The likelihood below treats the comparison vectors of different pairs as independent draws, a standard working assumption that ignores the dependence between pairs that share a record and the potential one-to-one structure of the matches. To find the parameters the first instinct is to maximize the likelihood over θ, but the log likelihood is a sum of logs of a sum so there is no closed form:

$$\ell(\theta) = \sum_{i=1}^{N} \log \left[\lambda \prod_k m_{k,\gamma_{ik}} + (1-\lambda) \prod_k u_{k,\gamma_{ik}} \right]$$

So instead, EM exploits the structure that the likelihood is a mixture and has the hidden variable M_i that, if we knew, would make it separable. So then we are interested in the joint of the comparison vectors and the known M_i given theta to maximize over θ, compared with the incomplete data version where we were only interested in the distribution of the comparison vectors given theta. Using selector exponents, we can write the joint using the chain rule and the previous definitions:

$$\begin{aligned} P(\gamma_i, M_i \mid \theta) &= P(M_i \mid \theta)\, P(\gamma_i \mid M_i, \theta) \\ &= \Big[\lambda \prod_k m_{k,\gamma_{ik}}\Big]^{M_i} \Big[(1-\lambda) \prod_k u_{k,\gamma_{ik}}\Big]^{1-M_i} \end{aligned}$$

Then taking the log and summing over pairs we get:

$$\begin{aligned} \ell_c(\theta) &= \sum_i \log P(\gamma_i, M_i \mid \theta) \\ &= \sum_i \Big[M_i \Big(\log \lambda + \sum_k \log m_{k,\gamma_{ik}} \Big) + (1-M_i) \Big(\log(1-\lambda) + \sum_k \log u_{k,\gamma_{ik}} \Big) \Big] \end{aligned}$$

But, of course, we don't have complete data. So EM instead uses the expected completed-data log likelihood, replacing M_i with p_i given some current θ'. Since ℓ_c is linear in each M_i, the expectation passes straight through where $\mathbb{E}[M_i] = P(M_i = 1 \mid \gamma_i, \theta') = p_i(\theta')$:

$$\begin{aligned} Q(\theta \mid \theta') &= \mathbb{E}_{M \sim P(\cdot \mid \gamma, \theta')} \big[\ell_c(\theta; M) \big] \\ &= \mathbb{E}_{M \sim P(\cdot \mid \gamma, \theta')} \Big[\sum_i \log P(\gamma_i, M_i \mid \theta) \Big] \\ &= \sum_i \mathbb{E}_{M_i \sim P(\cdot \mid \gamma_i, \theta')} \big[\log P(\gamma_i, M_i \mid \theta) \big] \\ &= \sum_i \sum_{M_i \in \{0,1\}} P(M_i \mid \gamma_i, \theta') \, \log P(\gamma_i, M_i \mid \theta) \end{aligned}$$

And expanding given the previous work gives:

$$\begin{aligned}
Q(\theta \mid \theta') &= \sum_i \sum_{M_i \in \{0,1\}} P(M_i \mid \gamma_i, \theta') \log P(\gamma_i, M_i \mid \theta) \\
&= \sum_i \Big[P(M_i = 1 \mid \gamma_i, \theta') \log P(\gamma_i, M_i = 1 \mid \theta) + P(M_i = 0 \mid \gamma_i, \theta') \log P(\gamma_i, M_i = 0 \mid \theta) \Big] \\
&= \sum_i \Big[p_i(\theta') \log \Big(\lambda \prod_k m_{k,\gamma_{ik}} \Big) + (1 - p_i(\theta')) \log \Big((1-\lambda) \prod_k u_{k,\gamma_{ik}} \Big) \Big] \\
&= \sum_i \Big[p_i(\theta') \Big(\log \lambda + \sum_k \log m_{k,\gamma_{ik}} \Big) + (1 - p_i(\theta')) \Big(\log(1-\lambda) + \sum_k \log u_{k,\gamma_{ik}} \Big) \Big]
\end{aligned}$$

Finally, note that we can switch summation order because they are finite sums and our notation with the gammas in the subscript means $\log m_{k,\gamma_{ik}} = \sum_j \mathbb{1}\{\gamma_{ik} = j\} \log m_{kj}$. For brevity, write $p_i = p_i(\theta')$, $N_m = \sum_i p_i$, and $N_u = \sum_i (1 - p_i)$.

$$\begin{aligned}
Q(\theta \mid \theta') &= \sum_i \Big[p_i(\theta') \Big(\log \lambda + \sum_k \log m_{k,\gamma_{ik}} \Big) + (1 - p_i(\theta')) \Big(\log(1-\lambda) + \sum_k \log u_{k,\gamma_{ik}} \Big) \Big] \\
&= \Big(\sum_i p_i \Big) \log \lambda + \Big(\sum_i (1 - p_i) \Big) \log(1-\lambda) + \sum_k \sum_i p_i \log m_{k,\gamma_{ik}} + \sum_k \sum_i (1 - p_i) \log u_{k,\gamma_{ik}} \\
&= \underbrace{N_m \log \lambda + N_u \log(1-\lambda)}_{Q_\lambda(\lambda)} + \sum_k \underbrace{\sum_j \Big(\sum_i p_i \, \mathbb{1}\{\gamma_{ik} = j\} \Big) \log m_{kj}}_{Q_{m,k}(m_{k\cdot})} \\
&\quad + \sum_k \underbrace{\sum_j \Big(\sum_i (1 - p_i) \, \mathbb{1}\{\gamma_{ik} = j\} \Big) \log u_{kj}}_{Q_{u,k}(u_{k\cdot})}
\end{aligned}$$

This form is nice for a couple of reasons: 1) p_i are constants because they are computed with θ', 2) log turns the products into sums so no term contains two parameters multiplied together, and 3) parameters for different comparisons k never appear in the same sum. This allows Q separability so it's a sum of blocks and each block is a function of one parameter group alone. Noting that a maximizer of a sum of functions of disjoint variables is found by maximizing each function on its own, we can decompose this into $1 + 2F$ independent problems (one for λ and one for each $m_{k\cdot}$, and $u_{k\cdot}$, also noting the PMF constraints).

This is all well and good, but let's not lose track of the final goal which is to maximize the likelihood function. We must show that maximizing Q, which is a nice case, never decreases ℓ. We can do this with one expectation magic trick and the use of Jensen's inequality. The proof is in Appendix A.

7 EM steps

The high level goal here is that for each EM iteration $t \geq 1$, where the first round using seeds is $t = 0$, we use $\theta^{(t-1)}$ to find $p_i(\theta^{(t-1)})$, input those values into Q, then estimate θ until $\max |\theta^{(t)} - \theta^{(t-1)}| < 10^{-4}$ or whatever convergence value is set in the Splink settings, or we reach max iterations (default 25).

EM consists of 1) the E-step (expectation step), and 2) the M-step (maximization step). Recall that $Q(\theta|\theta')$ is (noting these rewritings and simplifications: 1) $\log m_{k,\gamma_{ik}} = \sum_j \mathbb{1}\{\gamma_{ik} = j\} \log m_{kj}$, 2)

$p_i = p_i(\theta')$, $N_m = \sum_i p_i$, and $N_u = \sum_i (1 - p_i)$):

$$Q(\theta \mid \theta') = \underbrace{N_m \log \lambda + N_u \log(1-\lambda)}_{Q_\lambda(\lambda)} + \sum_k \underbrace{\sum_j \Big(\sum_i p_i \, \mathbb{1}\{\gamma_{ik} = j\} \Big) \log m_{kj}}_{Q_{m,k}(m_{k\cdot})}$$
$$+ \sum_k \underbrace{\sum_j \Big(\sum_i (1 - p_i) \, \mathbb{1}\{\gamma_{ik} = j\} \Big) \log u_{kj}}_{Q_{u,k}(u_{k\cdot})}$$

Next, to make this form more interpretable for the EM steps, note the following interpretations from each separable part. Given complete data, we should have the counts at level j of comparison k among matches to be $C^m_{kj} = \sum_i M_i \, \mathbb{1}\{\gamma_{ik} = j\}$. Taking the expectation gives:

$$E[C^m_{kj} \mid \gamma, \theta'] = \sum_i E[M_i \mid \gamma_i, \theta'] \, \mathbb{1}\{\gamma_{ik} = j\} = \sum_i p_i \, \mathbb{1}\{\gamma_{ik} = j\} = c^m_{kj}.$$

And it follows symmetrically for the u case. Within $Q_{u,k}(u_{k\cdot})$ and $Q_{m,k}(m_{k\cdot})$, we can write $c^m_{kj} = \sum_i p_i \, \mathbb{1}\{\gamma_{ik} = j\}$ and $c^u_{kj} = \sum_i (1 - p_i) \, \mathbb{1}\{\gamma_{ik} = j\}$. These represent the expected counts at level j of comparison k among matches and non-matches. Note also that $\sum_j c^m_{kj} = N_m$ and $\sum_j c^u_{kj} = N_u$ for every k, since each pair sits at exactly one level.

Now, starting at the E step, note that all we need to calculate is $p_i(\theta')$, which recall is the posterior:

$$p_i(\theta) = P(M_i = 1 | \gamma_i, \theta) = \frac{P(M_i = 1|\theta) P(\gamma_i | M_i = 1, \theta)}{P(\gamma_i | \theta)} = \frac{\lambda \prod_k m_{k,\gamma_{ik}}}{\lambda \prod_k m_{k,\gamma_{ik}} + (1-\lambda) \prod_k u_{k,\gamma_{ik}}}$$

This is all for step E, simply evaluating our posterior at the seed θ for $t = 0$ or evaluating it at $\theta^{(t-1)}$.

The final step is the M step, where we will maximize each separable component. First, note the simplest case which is lambda. Q_λ is unconstrained on $(0, 1)$ so simply:

$$\frac{\partial Q_\lambda}{\partial \lambda} = \frac{N_m}{\lambda} - \frac{N_u}{1-\lambda} = 0 \quad \Longrightarrow \quad \hat{\lambda} = \frac{N_m}{N_m + N_u} = \frac{1}{N} \sum_i p_i.$$

So, $\hat{\lambda}$ is the expected fraction of true matches given the current posterior. As a preview to the next section, in Splink the lambda estimate can also be calculated without the model's assumptions and without blocking during EM.

Next are the m estimates. For these we need a Lagrangian since we have the PMF constraint:

$$\mathcal{L}_{m,k}(m_{k\cdot}, \nu_k) = \sum_j c^m_{kj} \log m_{kj} - \nu_k \Big(\sum_j m_{kj} - 1 \Big).$$

Solving the first-order conditions gives:

$$\frac{\partial \mathcal{L}_{m,k}}{\partial m_{kj}} = \frac{c^m_{kj}}{m_{kj}} - \nu_k = 0 \quad \Longrightarrow \quad m_{kj} = \frac{c^m_{kj}}{\nu_k}, \qquad j = 1, \ldots, J_k,$$

$$\frac{\partial \mathcal{L}_{m,k}}{\partial \nu_k} = -\Big(\sum_j m_{kj} - 1 \Big) = 0 \quad \Longrightarrow \quad \sum_j m_{kj} = 1.$$

Substituting the first set of conditions into the second,

$$\sum_j \frac{c^m_{kj}}{\nu_k} = 1 \quad \Longrightarrow \quad \nu_k = \sum_j c^m_{kj} = \sum_i p_i \sum_j \mathbb{1}\{\gamma_{ik} = j\} = \sum_i p_i = N_m,$$

using $\sum_j \mathbb{1}\{\gamma_{ik} = j\} = 1$ since each pair occupies exactly one level. Substituting back,

$$\hat{m}_{kj} = \frac{c^m_{kj}}{N_m} = \frac{\sum_i p_i \, \mathbb{1}\{\gamma_{ik} = j\}}{\sum_i p_i}.$$

So, $\hat{m}_{kj}$ is the expected fraction of matches that have level j for comparison k, given the previous theta estimates. These are the parameters that Splink actually estimates with EM.

Because Splink leaves the null level out, its sums for comparison k run over the pairs that are non-null on k. The denominator is then the sum of p_i over those pairs and not N_m, which is the formula given in Section 8.4. The same adjustment applies to u.

Finally, the estimates for the u parameters are symmetrical:

$$\mathcal{L}_{u,k}(u_{k\cdot}, \mu_k) = \sum_j c^u_{kj} \log u_{kj} - \mu_k \Big(\sum_j u_{kj} - 1 \Big).$$

Solving the first-order conditions gives:

$$\frac{\partial \mathcal{L}_{u,k}}{\partial u_{kj}} = \frac{c^u_{kj}}{u_{kj}} - \mu_k = 0 \quad \Longrightarrow \quad u_{kj} = \frac{c^u_{kj}}{\mu_k}, \qquad j = 1, \ldots, J_k,$$

$$\frac{\partial \mathcal{L}_{u,k}}{\partial \mu_k} = -\Big(\sum_j u_{kj} - 1 \Big) = 0 \quad \Longrightarrow \quad \sum_j u_{kj} = 1.$$

Substituting the first set of conditions into the second,

$$\sum_j \frac{c^u_{kj}}{\mu_k} = 1 \quad \Longrightarrow \quad \mu_k = \sum_j c^u_{kj} = \sum_i (1 - p_i) \sum_j \mathbb{1}\{\gamma_{ik} = j\} = \sum_i (1 - p_i) = N_u,$$

using $\sum_j \mathbb{1}\{\gamma_{ik} = j\} = 1$ since each pair occupies exactly one level. Substituting back,

$$\hat{u}_{kj} = \frac{c^u_{kj}}{N_u} = \frac{\sum_i (1 - p_i) \, \mathbb{1}\{\gamma_{ik} = j\}}{\sum_i (1 - p_i)}.$$

So, $\hat{u}_{kj}$ is the expected fraction of non-matches that have the level j for comparison k, given the previous theta estimates.

The EM algorithm in Splink does many iterations of these calculations, using the seed $\theta^{(0)}$ then the previous run's $\theta^{(t-1)}$ as the current parameter values, until $\max |\theta^{(t)} - \theta^{(t-1)}| < 10^{-4}$ or $t = 25$ which implies EM didn't converge.

With EM covered, I'll now cover how Splink makes a few changes to improve EM stability and make EM feasible for large datasets.

8 Splink workflow

Splink estimates u by random sampling and λ from deterministic match rules, and then holds u fixed during EM. By default λ is not fixed. Each EM session starts from the deterministic estimate, adjusted for blocking, and updates it. The session value is not saved, so prediction uses the deterministic

estimate. The point of these external estimates is to constrain the mixture. With u and λ free, EM can ascend toward any two-component partition of the pairs that explains the joint frequencies of comparison vectors, and such a partition need not be match/non-match (e.g., same household/different household, clean data/messy data, etc.). Fixing u to the non-match distribution pins one component and starting λ near its true value anchors the other, so the only partition consistent with the data would be match/non-match (given the CI assumption holds), and the stationary point EM reaches is an estimate of m for that partition rather than for some other split.

8.1 Prior from deterministic rules

The way that lambda is estimated depends on 1) a set of deterministic rules which are strict enough such that every pair satisfying the rules is assumed to be a true match and 2) a recall estimate that represents the number of true matches the deterministic rules capture out of all the true matches. The lambda estimate then uses those two values, and the total number of pairs possible, to estimate the total matches over total pairs (which is lambda). Formally, with two datasets A and B of size $|A|$ and $|B|$:

$$\hat{\lambda} = \frac{\text{\# of pairs satisfying strict rules/recall}}{|A| \cdot |B|}$$

For the deduplication of a single dataset A, the denominator is the number of distinct pairs $|A|(|A|-1)/2$.

The maintainers of Splink mention that the model is robust to a misestimated recall. The first reason is because all pairs get the same lambda so the order is unaffected by a misestimation, only the threshold is affected. The second reason depends on the effect size. Recall the total match weight for a pair (odds in log space):

$$w_i = \underbrace{\log_2 \frac{\lambda}{1-\lambda}}_{\text{prior weight}} + \sum_k \log_2 \frac{m_{k,\gamma_{ik}}}{u_{k,\gamma_{ik}}}$$

and recall that $\lambda \propto 1/\text{recall}$ as above. So guessing recall wrong by a factor of r makes the estimated $\hat{\lambda}$ wrong by a factor of $1/r$. Then since lambda is very small we know $\lambda/(1-\lambda) \approx \lambda$. So then the prior weight above, with the recall misestimate, can be rewritten as $\log_2 \lambda - \log_2 r$. So then if we guess 0.85 and the truth is 0.6 then the prior weight is ~0.5 bits too low. If the truth is 0.95, then the prior weight is ~0.16 bits too high. Given that an exact match on DOB could add 10+ bits, a mis-estimate in recall will give a slight miscalibration in match probability.

8.2 u from random sampling

The next pre-EM estimated parameter is u. The estimation strategy depends on λ being very small, typically ~10^{-8}. Given that, if we take a large random sample of x records from the left and right dataset (or one sample from the same dataset if it's a deduplication, not a merge), a very very small proportion will be matches. So, we can assume that every pair is a non-match and get a very good estimate. The way it actually works in Splink is that a number of records is drawn by Bernoulli sample with a proportion determined by the max_pairs set in the config using *proportion = sqrt(max_pairs / (n_left * n_right))* (to get the proportion that results in the correct number of max pairs). When it's dedup, it inverts *max_pairs = r(r-1)/2* to get r which is the number of records to pair to get max_pairs without order or self-pairing. Splink then takes the full cross product between the two samples (or cross product of one sample without self-merge in the dedup case). Afterwards, Splink converts each pair into its comparison vector (γ_{ik}, the level of comparison k attained by pair i, with $\gamma_{ik} = 1$ when either record is null on the field). Let $\mathcal{S}$ denote the set of sampled pairs, $|\mathcal{S}| \approx N$ (the

max_pairs target), and let $\mathcal{S}_k = \{i \in \mathcal{S} : \gamma_{ik} \neq 1\}$ be the pairs on which comparison k is non-null. The estimated u values are then simply level shares among non-null sampled pairs:

$$\hat{u}_{kj} = \frac{\sum_{i \in \mathcal{S}_k} \mathbb{1}\{\gamma_{ik} = j\}}{|\mathcal{S}_k|}, \qquad j \in \{2, \ldots, J_k\},$$

8.3 m from blocked EM

Finally, the main star of the show are the m parameter estimations. First, consider the case where we used every record from every dataset to run EM. For two files of ten million records each, the cross product results in 10^{14} pairs which need to be built into comparison vectors (scored on JW, DL, etc.) and aggregated into counts. Even optimistically, this is an unrealistic amount of compute. To solve this issue, Splink uses blocking for EM. Statistically, this is also more efficient since a pair that disagrees on every comparison feature won't be a match anyway. Blocking in this case means creating a set of multiple blocking sets, where each blocking set runs EM once to produce m estimates. The catch is twofold: 1) if you block on a variable (say first name), an unadjusted prior (λ) will be too low since the prior needs to account for a matching first name and 2) we cannot estimate an m parameter for a variable that we are blocking on. Therefore, we need to adjust the prior to be reasonable within a blocked set of pairs and need to run multiple EM runs with different blocking variables to estimate all the m parameters. Also, running multiple EM sessions with different blocks helps to see if the blocks themselves are dramatically changing the m parameter estimation.

The main adjustment needed then is to correctly adjust the prior within an EM block to account for the change in distribution from the blocking. However, we have a chicken-and-egg situation where we haven't quite learned the parameters needed to adjust the prior. Therefore, Splink will use the seed values or the running estimates of the parameters so far. In addition, Splink will automatically use the exact-level parameters from the category to adjust it. So for example, if the first EM pass is just blocked on exact date of birth, then Splink will use the seed parameters from the dob (or the in-progress learned parameters if it is not the first EM pass) to adjust the prior. However, during each m step of EM, the prior is updated to be consistent with the blocked distribution (the deterministic lambda is the actual one used during prediction, these ones are changing to support the m estimation). Therefore the adjusted block prior is only needed to point EM in the best general direction to start. Thus, it is likely a bad idea to fix the prior during EM because the prior must then rely on previous or seed estimates to adjust the total sample prior. This is especially true if some of the feature levels are complex (in that they use multiple variables in non-trivial ways), since blocking on any variable used in that feature will not adjust the prior correctly. Splink will only adjust the prior when blocked on a variable for which there exists a level in the comparisons for that variable with a strict col_l = col_r or an AND of them (and then all the cols in the AND would need to be a part of the blocking specification). In the cases where the blocking adjustment does take place, the adjusted prior becomes the following, noting that we once again rely on conditional independence for the result and eligible means Splink actually adjusts for the variable being blocked based on the above criteria:

$$\pi_{\text{session}} = \pi \times \prod_{k \in \text{blocked and eligible}} K_{k,\text{exact}} \qquad \left(\text{so} \quad \lambda_{\text{session}} = \frac{\pi_{\text{session}}}{1 + \pi_{\text{session}}}\right),$$

or to put it into log odds form:

$$\log_2 \pi_{\text{session}} = \log_2 \pi + \sum_{k \in \text{blocked and eligible}} \log_2 K_{k,\text{exact}}$$

8.4 Term frequency adjustments

The final nuance to Splink's approach is term-frequency (TF) adjustments, which allow for common names to count less on agreement than rarer names. TF adjustments are used in training and prediction. I'll cover training here and prediction below. To understand how TF adjustment plays a part in the training process consider the restatement of the model below.

In Splink's library comparisons, TF adjustments only happen for a variable when a pair has an exact match on it. A custom comparison can attach a TF adjustment to other levels, which this section does not cover. For the sake of explanation, consider the first name as the variable that we will TF adjust. Let's first focus on how the u parameter changes in TF adjustment. For two datasets A and B with the first name in a pair from A being a and the first name in a pair from B being b, we can write the following:

$$u_{\text{exact}} = P(\text{first name agree} \mid M = 0) = \sum_v u_{\text{exact}}(v), \qquad u_{\text{exact}}(v) = P(a = v,\ b = v \mid M = 0).$$

Then using the independence assumption from the fact that the two records belong to different people so their first names are independent (breaking with siblings or relatives, which is a fine exception since almost all random pairs are unrelated):

$$P(a = v,\ b = v \mid M = 0) = P(a = v \mid M = 0)\, P(b = v \mid M = 0)$$

But since we know that a random pair is normally a match with probability around 10^{-8}, we can make the assumption that a random record drawn from the file is, to that precision, approximately:

$$P(a = v \mid M = 0) \approx p_A(v), \qquad P(b = v \mid M = 0) \approx p_B(v)$$

So then:

$$u_{\text{exact}}(v) \approx p_A(v) p_B(v)$$

With $p_A(v)$ representing the probability of seeing value v in the first names of dataset A, and similarly for B. Then, summing over values recovers the ordinary exact level $u_{\text{exact}} = \sum_v u_{\text{exact}}(v) = \sum_v p_A(v)\, p_B(v)$.

Now, we hit more barriers with TF adjustment for m. The conceptual goal is to split the exact level defined in the first name comparison list into sub-levels such that each sub-level represents "exact and the shared value is some v". So starting with:

$$m_{\text{exact}} = P(\text{the two records' values agree} \mid \text{the pair is a match})$$

We can use the sub-level framework to re-write as:

$$m_{\text{exact}} = \sum_v m_{\text{exact}}(v), \qquad m_{\text{exact}}(v) = P(a = v,\ b = v \mid M = 1)$$

Let $q(t) = P(T = t|M = 1)$ where T represents a matched person's first name. We can then decompose the above probability into:

$$\begin{aligned} m_{\text{exact}}(v) = P(a = v,\ b = v \mid M = 1) &= \sum_t P(T = t \mid M = 1)\, P(a = v,\ b = v \mid T = t,\ M = 1) \\ &= \sum_t q(t)\, P(a = v,\ b = v \mid T = t) \end{aligned}$$

Then since $t = v$ will dominate the sum, assuming that coincidental identical typos that happen to each side are negligible, we can approximate it as:

$$m_{\text{exact}}(v) \approx q(v)\, c(v), \qquad c(v) = P(\text{both records spell it correctly} \mid \text{true name } v)$$

So then we could turn it back into the pooled parameter with $m_{\text{exact}} = \sum_v q(v)\, c(v)$, which can be interpreted as the average recording accuracy, weighted by how many matched people have each name. Here we use the assumption that recording accuracy doesn't depend on name, so that $c(v) = c$ for every v. This allows us to make the nice simplification:

$$m_{\text{exact}} = \sum_v q(v)\, c = c \sum_v q(v) = c$$

Which allows for:

$$m_{\text{exact}}(v) \approx q(v)\, c(v) = q(v)c = m_{\text{exact}} q(v)$$

The necessity here is that EM can't estimate $c(v)$ separately for each name. It only has one m_{exact} to fit, so it uses the accuracy assumption.

The Bayes factor for agreeing on v is then the ratio of the two "sub-level" probabilities:

$$K_{\text{exact}}(v) = \frac{m_{\text{exact}}(v)}{u_{\text{exact}}(v)} \approx \frac{m_{\text{exact}}\, q(v)}{p_A(v)\, p_B(v)}.$$

This still involves three different name distributions: q (names among true matches), p_A and p_B. Splink can't observe q, since it doesn't know which pairs are matches. So we make an assumption that matchability (i.e., having a match in the pairing universe) is independent of the actual name. Then, q is just the name distribution among the A records that have a partner, which is equal to the name distribution among the A records, by the assumption. So $q \approx p_A$ and we get:

$$K_{\text{exact}}(v) = \frac{m_{\text{exact}}\, p_A(v)}{p_A(v)\, p_B(v)} = \frac{m_{\text{exact}}}{p_B(v)}.$$

One way to interpret this Bayes factor is to fix one record named Maria in dataset A and ask how surprising it is that a record in dataset B also says Maria. If it's the true partner record, then it says it with probability m_{exact} given the recording assumptions. If it's not her true partner, then it will say Maria with $p_B(\text{Maria})$, thus creating the correct odds accounting for name frequency. Ideally we would have p_B but instead we approximate it by pooling the two datasets together and finding the frequency of each name, notated by $tf(v)$ where $|N_A|$ is the number of non-null first names in A and n_v is the number of records across A and B that have v as their first name:

$$p_B(v) \approx tf(v) = \frac{n_v}{|N_A| + |N_B|}$$

We pool both datasets here, because we could make an identical argument, starting with fixing a record in B and then finding the tf in A. So in reality, we need $p_B(v) \approx tf(v) \approx p_A(v) \approx q(v)$, which requires matchability to be independent of name in both datasets, so that the two have similar name distributions (noting that this is separate from the recording accuracy assumption, which states that, conditional on true match, both records write the name identically – the matching independence is that the chances of having a partner in the pair universe don't depend on the name). When the name distributions differ, the pooled frequency is a size-weighted average of p_A and p_B and is closest to that of the larger dataset.

So we can write the final approximation that Splink uses:

$$K_{\text{exact}}(v) \approx \frac{m_{\text{exact}}\,\text{tf}(v)}{\text{tf}(v)^2} = \frac{m_{\text{exact}}}{\text{tf}(v)}.$$

The evidence from agreeing on v is the chance that a true match agrees, divided by how common v is.

However, Splink doesn't build a separate level for each name. It keeps the pooled exact level, with its one m_{exact} and one u_{exact}, and multiplies that level's Bayes factor by a correction for each pair:

$$K_{\text{exact}}(v) = \underbrace{\frac{m_{\text{exact}}}{u_{\text{exact}}}}_{\text{pooled level}} \cdot \underbrace{\frac{u_{\text{exact}}}{\text{tf}(v)}}_{T(v)} = \frac{m_{\text{exact}}}{\text{tf}(v)}.$$

Two things follow. First, u_{exact} cancels, so at the exact level a pair's score depends only on m_{exact} and $\text{tf}(v)$. Second, the pooled Bayes factor is the same as the TF adjusted Bayes factor for a name with frequency exactly u_{exact}. Names with $\text{tf}(v) < u_{\text{exact}}$ get $T > 1$ and gain weight, and more common names get $T < 1$ and lose weight.

In training during EM, the adjusted Bayes factor enters only in the E-step. Each pair's posterior odds pick up the factor T for every active comparison on which it agrees exactly on a TF column. So the odds and probability become:

$$O_i^{(t)} = \frac{\lambda}{1-\lambda} \prod_k \frac{m_{k\gamma_{ik}}^{(t-1)}}{u_{k\gamma_{ik}}}\, T_{k\gamma_{ik}}(i), \qquad p_i^{(t)} = \frac{O_i^{(t)}}{1 + O_i^{(t)}}.$$

Because T is built from a fixed u and a count, it is a constant for each pair and is never updated. Pairs that don't agree exactly on any active TF column have $T = 1$ and get the same posterior as they would without TF. Since two pairs with the same comparison vector can now have different posteriors, EM can no longer collapse pairs into agreement patterns, and every pair is kept as its own row. When EM is run with estimate_without_term_frequencies=True, Splink groups the session's pairs by their comparison vector over the active comparisons, such that pattern g holds n_g pairs. It then calculates an equivalent version of the M-step that we derived above:

$$m_{kj} = \frac{\sum_{g:\, g_k = j} n_g\, p_g}{\sum_{g:\, g_k \neq \text{null}} n_g\, p_g}$$

But with TF on, we have to go back to the original formula, $m_{kj} = \dfrac{\sum_{i:\, \gamma_{ik}=j} p_i}{\sum_{i:\, \gamma_{ik}\neq\text{null}} p_i}$, which is slower. Note that the lambda updates similarly change.

The M-step formula is unchanged, but it is fed the new posteriors, and the same posteriors weight every comparison. So TF on first name changes the estimates for county, gender and the other names as well as its own m_{exact} (and λ, if it isn't fixed). What TF does not change is u since it is fixed by random sampling with TF removed, the TF values themselves, the population and blocking-adjusted priors (which use the plain m/u), and the number of parameters, since there is still only one m_{exact} per comparison. A column used in a session's blocking rule is deactivated for that session, and its TF goes with it.

8.5 Prediction

After training, the parameters EM produced are used for prediction. No further estimation happens here – the m's, u's and prior are fixed, and each candidate pair is scored once. Generally, prediction

is fairly simple. It starts by generating candidate pairs with the prediction blocking rules, which are specified separately from the training rules and are made to be as loose as possible within computational limits, since their job is to capture as many true matches as possible rather than to isolate informative subsets for EM. Splink takes the union across rules, skipping any pair already produced by an earlier rule so that no pair is scored twice. A pair produced by no rule is never compared, so a true match missed by every blocking rule is lost at this stage.

Every candidate pair is then compared on all comparisons defined. Each comparison places the pair at a level given each comparison variable, and the pair's match weight (log odds, with units of bits) is the prior plus the sum of the level weights, with the TF factor added wherever the pair agrees exactly on a TF column:

$$\text{match_weight}_i = \log_2 \frac{\lambda}{1-\lambda} + \sum_k \Big[\log_2 \frac{m_{k\gamma_{ik}}}{u_{k\gamma_{ik}}} + \log_2 T_{k\gamma_{ik}}(i) \Big], \qquad p_i = \frac{2^{\text{match_weight}_i}}{1 + 2^{\text{match_weight}_i}}.$$

Here λ is the population prior, not the blocking-adjusted prior of any training session, and $T = 1$ at every level without TF.

9 Extensions

This section describes three extensions I made to the Splink workflow. The first is a sampling scheme for EM training when individuals have several ranked observations, the second runs prediction in parallel and the third enforces one-to-one links after prediction.

9.1 Sampling with multiple ranked observations

EM training on the full pool of candidate records can be too slow, so I train on a subsample. An individual can have several recorded versions of its identifying fields, such as name variants, which I call ranked observations and number by rank with lower ranks preferred. The pool holds one row for each ranked observation of an individual, so an individual with three available ranks contributes three rows. The subsample should reproduce the rank composition of the pool, so that EM trains on the same distribution of data quality that it scores at prediction, and it should hold at most one row per individual, because EM treats pairs as independent draws.

Requirements

Let the pool hold n individuals and N rows. Individual i has a set S_i of available ranks of size R_i, so that $N = \sum_i R_i$. Let f be the target fraction of individuals in the training sample, for example $f = 0.15$. The sampling scheme should meet three requirements.

1. **Equal row inclusion.** Every row of the pool enters the sample with the same probability. This preserves the rank composition in expectation.

2. **One row per individual.** The sample holds at most one row of each individual. This removes the dependence between rows of the same individual.

3. **Calibrated size.** The expected number of sampled individuals is fn.

A Bernoulli sample of rows meets the first requirement alone, and any scheme that draws one row per individual meets the second alone. Meeting both requires selection probabilities that vary across individuals with the number of ranks and are equal across rows.

Simpler schemes

Three simpler schemes each fail one of the first two requirements.

The first is a Bernoulli sample of rows at rate f. It meets the first requirement but not the second, since both the rank 1 and the rank 2 rows of an individual can be drawn. EM would count the resulting pairs as independent observations, which would overstate the effective sample size and make the estimates less precise than the number of pairs suggests.

The second draws the same Bernoulli sample and then keeps one sampled row per individual at random. It meets the second requirement but not the first. An individual with R_i ranks has at least one row drawn with probability $1-(1-f)^{R_i}$, and each of its rows is equally likely to be the one kept, so the row inclusion probability is

$$\Pr\left(\text{row } (i,r) \text{ sampled}\right) = \frac{1-(1-f)^{R_i}}{R_i}.$$

This equals f for a singleton and falls as R_i grows. With $f = 0.15$ it is 0.139 for two ranks and 0.129 for three, so rows of individuals with several ranks are underrepresented. An individual with more ranks is more likely to be kept, but less than in proportion to its number of ranks, which is what equal row inclusion requires. Rejection weights could correct this, but that amounts to designing an unequal probability scheme, which is what the scheme below does directly.

The third keeps every individual and draws one of its rows with probability proportional to the availability of each rank in the pool. Let $a_r = n_r/n$ be the share of individuals for whom rank r is available, where n_r is the number of such individuals. The rule is

$$\Pr\left(\text{rank } r \text{ drawn} \mid S_i\right) = \frac{a_r}{\sum_{j \in S_i} a_j}, \qquad r \in S_i.$$

It meets the second requirement but not the first. A singleton has $S_i = \{1\}$ and contributes its rank 1 row with probability one. Singletons are a large share of the pool, which holds the rank 1 share of the sample well above its pool share n_1/N whatever the values of a_r.

The third scheme points to a general limit. No scheme can keep every individual and also meet the first two requirements unless all individuals have the same number of ranks. If every individual is kept with exactly one row, the inclusion probabilities of its rows sum to one, so a common row probability ρ would require $\rho R_i = 1$ for every i. The scheme below meets the first two requirements and gives up the inclusion of every individual. The cost is small when training on the full pool is too slow in any case.

Sampling scheme

The scheme uses the following notation.

Symbol	Meaning
n	number of individuals in the pool
N	number of rows in the pool, $N = \sum_i R_i$
S_i	set of ranks available for individual i
R_i	number of ranks available for individual i
$R_{\max}$	largest number of ranks of any individual
$\bar{R} = N/n$	average number of ranks per individual
n_r	number of individuals with rank r available
f	target fraction of individuals in the sample
$\rho = f/\bar{R}$	row inclusion probability
h_i	uniform draw for individual i

The row inclusion probability is $\rho = f/\bar{R}$. If every individual were a singleton, ρ would equal f and the scheme would be a Bernoulli sample of rows at rate f. Dividing by $\bar{R}$ offsets the extra rows that individuals with several ranks contribute, so that the expected fraction of individuals sampled remains f.

Each individual receives one draw h_i that is treated as uniform on $[0, 1)$. In practice $h_i = (\text{hash}(i, \text{seed}) \bmod M)/M$ for a large modulus M such as 10^9, so the sample is reproducible given the seed and the value 1 is never reached. The scheme applies four steps to each individual i.

1. Compute h_i.
2. Sort S_i in ascending order and index its elements $1, \ldots, R_i$.
3. If $h_i < \rho R_i$, keep the individual and select the rank at position $\lfloor h_i/\rho \rfloor + 1$.
4. Otherwise drop the individual.

The single draw decides both whether the individual is kept and which rank is selected. The scheme divides the unit interval into R_i rank slices of width ρ and one drop slice,

$$[0, 1) = \bigcup_{j=1}^{R_i} [(j-1)\rho,\ j\rho) \ \cup\ [\rho R_i,\ 1),$$

and selects position j when h_i falls in slice j. The floor function returns the correct position because $h_i \in [(j-1)\rho, j\rho)$ holds exactly when $\lfloor h_i/\rho \rfloor = j - 1$. The intervals are half-open so that they cover $[0, 1)$ without gaps or overlaps. The table shows the division for individuals with one, two, three and R ranks.

Ranks	Rank slices	Drop slice	Probability individual is kept	Probability of each row
1	$[0, \rho)$	$[\rho, 1)$	ρ	ρ
2	$[0, \rho)$, $[\rho, 2\rho)$	$[2\rho, 1)$	2ρ	ρ
3	$[0, \rho)$, $[\rho, 2\rho)$, $[2\rho, 3\rho)$	$[3\rho, 1)$	3ρ	ρ
R	R slices of width ρ	$[\rho R, 1)$	ρR	ρ

The rank slices of every individual must fit in the unit interval. This feasibility condition is $\rho R_{\max} \leq 1$, or in terms of the target fraction

$$f \leq \frac{\bar{R}}{R_{\max}}.$$

The probability of keeping an individual is ρR_i, which rises with the number of ranks and cannot exceed one. A target fraction above the bound would require the individuals with the most ranks to be kept with probability greater than one. At the bound those individuals are kept with certainty. The bound measures how much of the pool the scheme must give up in exchange for the first two requirements.

Properties

Assume that the draws h_i are independent and uniform on $[0, 1)$ and that the feasibility condition holds. The scheme then meets the three requirements.

Equal row inclusion. Take any individual i and any rank $r \in S_i$. Conditional on the individual being kept, h_i/ρ is uniform on $[0, R_i)$, so each position is selected with probability $1/R_i$. The row inclusion probability is therefore

$$\Pr\left(\text{row } (i,r) \text{ sampled}\right) = \underbrace{\Pr(i \text{ kept})}_{\rho R_i} \cdot \underbrace{\Pr(\text{rank } r \text{ selected} \mid i \text{ kept})}_{1/R_i} = \rho.$$

The probability of keeping an individual rises with R_i and the probability of selecting a given rank falls with R_i, and the two effects cancel. The expected number of sampled rows of rank r is ρn_r out of an expected total of ρN, which reproduces the pool share n_r/N.

One row per individual. The draw h_i falls in exactly one slice, so an individual contributes one row or none. Rows of different individuals are selected independently because the draws are independent.

Calibrated size. The expected number of sampled individuals is

$$\mathbb{E}[\text{individuals kept}] = \sum_{i=1}^{n} \rho R_i = \rho N = fn.$$

Because the hash takes values on a grid of spacing $1/M$, these probabilities hold up to an error of order $1/M$.

Relation to survey sampling

Individuals act as clusters that are selected independently with probability proportional to size, where size is the number of ranks, and one row is then drawn with equal probability within each selected cluster. The two probabilities multiply to a constant, which makes the sample self-weighting. Valliant, Dever and Kreuter (2018) treat the general case in which the number of units drawn within a cluster is a parameter. As far as I can tell, the scheme above is the special case with one unit per cluster and uniform selection within the cluster, and their feasibility condition matches the one above.

9.2 Parallel prediction

One extension I made to the canonical Splink workflow, because the set of prediction pairs to-be-scored can run into the 10s of billions, is to run prediction in parallel. When pairing dataset A and dataset B, I split the largest dataset, call it A for now, into shards, then I launch a given number of workers. Each worker reads a shard of A and the full dataset B (hence where the memory speed tradeoff comes from). Then it produces a now smaller set of to-be-scored pairs, which it scores accordingly. Afterwards, the sharded scored pairs are merged together, then I proceed with the 1:1 enforcement or the entity resolution. The only thing that I had to change in the Splink workflow was that I calculate the TF for each variable that uses TF adjustments before sharding, then feed each shard the pre-sharded TF table which it then uses for TF adjustments.

9.3 One-to-one comparisons

Splink scores every candidate pair on its own, so nothing prevents one person from scoring above the match threshold with several persons in the other file. I call a person in the file being linked an entity and a person in the reference file a reference person. I treat both files as free of duplicates at the person level, which means that an entity can correspond to at most one reference person and a reference person to at most one entity. This section describes how I impose that constraint after prediction. I have implemented two methods, a greedy rule based on mutual best links and the linear sum assignment problem (LSAP) of Jaro (1989).

Candidate edges

Let p_{ab} denote the match probability between entity a and reference person b. Either side can enter prediction as several ranked versions of the same person, such as name variants, so a pair of persons can be scored more than once. I define p_{ab} as the highest match probability over all version pairs of a and b, with ties broken in favor of the lower ranks. The candidate set keeps the pairs of persons that reach the match threshold τ,

$$E = \{(a, b) \mid p_{ab} \geq \tau\}.$$

Collapsing the versions before the assignment keeps the constraint at the person level. Without this step two versions of one entity could each claim a different reference person. The set E defines a bipartite graph with entities on one side and reference persons on the other. A one-to-one assignment is a subset of E in which every entity and every reference person appears at most once.

Mutual best links

The first method accepts an edge when it is the highest-probability remaining edge for both of its endpoints, removes the two matched persons and their other edges from the graph, and repeats until no edges remain. The result is the greedy matching that takes edges in descending order of probability. Splink implements the rule in its one-to-one clustering algorithm, which merges two records when each is the best available link of the other and the merge would not place two records from the same duplicate-free dataset in one cluster. With two duplicate-free files this is the rule above.

Splink exposes the algorithm through the method cluster_using_single_best_links, which works on the record identifiers given to the linker. In my work, an identifier refers to one ranked version of an entity, so the method would treat two versions of the same entity as separate records and could link that entity to two reference persons. I therefore call the underlying function directly on node and edge tables built from E, in which each entity and each reference person is a single node. Ties in probability are broken by the lower identifier, which makes the assignment reproducible.

Linear sum assignment

The second method chooses the assignment with the largest total match probability. With $x_{ab} = 1$ when the edge (a, b) is accepted and $x_{ab} = 0$ otherwise, it solves

$$\begin{aligned} \max_{x} \quad & \sum_{(a,b)\in E} p_{ab}\, x_{ab} \\ \text{subject to} \quad & \sum_{b} x_{ab} \leq 1 \text{ for all } a, \qquad \sum_{a} x_{ab} \leq 1 \text{ for all } b, \qquad x_{ab} \in \{0, 1\}. \end{aligned}$$

Jaro (1989) showed that one-to-one assignment in record linkage can be written as an LSAP, for which efficient solvers exist. I follow the version in Enamorado, Fifield and Imai (2019, Supplementary Information S5), whose FS model specification is close to that of Splink. They sum match probabilities over the pairs above a threshold and give absent pairs a probability of zero. If the probabilities are calibrated, the objective is the expected number of correct links, so the solution maximizes that expectation among one-to-one assignments.

The objective separates across the connected components of the candidate graph, so I solve each component on its own. A component with a single edge needs no solver and is accepted directly, and most components are of this kind. For the others I build a matrix of match probabilities with zeros for absent edges and pass it to the scipy function linear_sum_assignment. An assignment that lands on a zero cell is not an edge and is discarded.

Comparison of the two methods

The two methods differ only in what they optimize within a component. Mutual best applies a local rule and never gives up an edge that is the best link for both of its endpoints. The LSAP optimizes the total and gives up such an edge when doing so raises the sum. Suppose entity a_1 has candidates b_1 at 0.90 and b_2 at 0.85, and entity a_2 has the single candidate b_1 at 0.80. Mutual best accepts (a_1, b_1), which leaves a_2 without a candidate, for a total of 0.90 and one link. The LSAP accepts (a_1, b_2) and (a_2, b_1) for a total of 1.65 and two links.

The methods agree on components with a single edge and on components where no accepted edge blocks a better total. They diverge only when entities compete for the same reference persons with similar probabilities. Enamorado, Fifield and Imai note that keeping the highest-probability match for each record may introduce bias in this setting, where several candidates have fairly large match probabilities. When this competition is largely absent from the data, the two methods give nearly the same assignment.

A model-based alternative

The LSAP returns a set of links but leaves the match probabilities unchanged, since they come from a model that ignores the constraint. McVeigh and Murray (2017) build the constraint into estimation. As I read it, their method starts from Fellegi-Sunter weights, estimates a penalized version of the model in which the matching must be one-to-one, and then samples from an approximate posterior to obtain match probabilities that respect the constraint. I did not adopt it because it adds a second model on top of the Splink model and the accompanying code is written in Julia.

Appendix A. Maxing Q never decreases ℓ

This appendix shows that an EM update never decreases the observed-data log likelihood, which is Theorem 1 of Dempster, Laird and Rubin (1977).

To get to use Jensen's inequality, we first have to use the most classic trick of multiplying by a fancy 1. In this case, we choose some arbitrary distribution $q_i(M_i)$ over $M_i \in \{0, 1\}$ with the restriction that $q_i(0) + q_i(1) = 1$ to ensure it is a valid PMF (and $q_i(M_i) > 0$ to ensure division is defined). Then, marginalizing out M_i and multiplying by 1, we have:

$$\begin{aligned} P(\gamma_i|\theta) &= \sum_{M_i} P(\gamma_i, M_i|\theta) \\ &= \sum_{M_i} q_i(M_i)\, \frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)} \end{aligned}$$

But now we have an expectation! Namely, $\mathbb{E}_{M_i \sim q_i}[\frac{P(\gamma_i, M_i|\theta)}{q_i(M_i)}]$. Then applying Jensen's inequality on the concave function log gives:

$$\begin{aligned} \log P(\gamma_i|\theta) = \log \sum_{M_i} q_i(M_i)\, \frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)} &= \log \mathbb{E}\Big[\frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)}\Big] \\ \geq \mathbb{E}\Big[\log \frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)}\Big] &= \sum_{M_i} q_i(M_i)\, \log \frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)} \end{aligned}$$

We then look at the end of the equality to split the log of the ratio into a difference:

$$\begin{aligned}\sum_{M_i} q_i(M_i)\ \log \frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)} &= \sum_{M_i} q_i(M_i) \log P(\gamma_i, M_i \mid \theta) - \sum_{M_i} q_i(M_i) \log q_i(M_i) \\ &= \mathbb{E}_{M_i \sim q_i}[\log P(\gamma_i, M_i|\theta)] + H(q_i)\end{aligned}$$

We end up with the expected complete data log likelihood for i under q_i and $H(q_i)$ which is the entropy (the important part here is that it does not depend on θ and so as we eventually maximize it will stay a constant). Summing over all pairs we get:

$$\ell(\theta) = \sum_i \log P(\gamma_i|\theta) \geq \sum_i \mathbb{E}_{q_i}\left[\log P(\gamma_i, M_i \mid \theta)\right] + \sum_i H(q_i)$$

The important part to note here is that the above equation describes a family of lower bounds that holds for any choice of q_i. If we choose q_i to be the posterior $P(M_i|\gamma_i, \theta')$, then we get:

$$\ell(\theta) \geq \sum_i \mathbb{E}_{P(M_i|\gamma_i,\theta')}\left[\log P(\gamma_i, M_i \mid \theta)\right] + \sum_i H(q_i) = Q(\theta|\theta') + H(q)$$

With this choice, we can show that the bound is tight at $\theta = \theta'$. The important point here is that the KL divergence of two distributions is 0 if and only if the two distributions are equal. Thus, we know that $KL(P(M_i|\gamma_i, \theta')\ ||P(M_i|\gamma_i, \theta')) = 0$. And that by definition of KL divergence $KL(f||g) = \sum_i f(x_i) \log \frac{f(x_i)}{g(x_i)}$. Then, differencing $\ell(\theta) - (Q(\theta|\theta') + H(q))$, and using the fact that q_i sums to 1 we get (and starting with the pre-decomposed form):

$$\begin{aligned}&\log P(\gamma_i \mid \theta) - \sum_{M_i} q_i(M_i) \log \frac{P(\gamma_i, M_i \mid \theta)}{q_i(M_i)} \\ &= \log P(\gamma_i \mid \theta) - \sum_{M_i} q_i(M_i) \log \frac{P(\gamma_i \mid \theta)\, P(M_i \mid \gamma_i, \theta)}{q_i(M_i)} \\ &= \log P(\gamma_i \mid \theta) - \sum_{M_i} q_i(M_i) \log P(\gamma_i \mid \theta) - \sum_{M_i} q_i(M_i) \log \frac{P(M_i \mid \gamma_i, \theta)}{q_i(M_i)} \\ &= \log P(\gamma_i \mid \theta) - \log P(\gamma_i \mid \theta) \sum_{M_i} q_i(M_i) - \sum_{M_i} q_i(M_i) \log \frac{P(M_i \mid \gamma_i, \theta)}{q_i(M_i)} \\ &= \log P(\gamma_i \mid \theta) - \log P(\gamma_i \mid \theta) \cdot 1 - \sum_{M_i} q_i(M_i) \log \frac{P(M_i \mid \gamma_i, \theta)}{q_i(M_i)} \\ &= \sum_{M_i} q_i(M_i) \log \frac{q_i(M_i)}{P(M_i \mid \gamma_i, \theta)} \\ &= \mathrm{KL}\left(q_i \,\|\, P(\cdot \mid \gamma_i, \theta)\right)\end{aligned}$$

So when we set $q_i = P(M_i|\gamma_i, \theta')$ and choose $\theta = \theta'$ we get the difference to be $\mathrm{KL}(P(\cdot \mid \gamma_i, \theta') \,\|\, P(\cdot \mid \gamma_i, \theta')) = 0$. Thus, the inequality $\ell(\theta) \geq Q(\theta|\theta') + H(q)$ is tight at $\theta = \theta'$.

Taking a step back, this shows that $\ell(\theta') = Q(\theta'|\theta') + H(q)$. So if we maximize to get $\theta'' = \arg\max_\theta Q(\theta \mid \theta')$, which is possible because Q is separable as shown in the EM theory section, then we can write:

$$\ell(\theta'') \geq Q(\theta'' \mid \theta') + H(q) \geq Q(\theta' \mid \theta') + H(q) = \ell(\theta')$$

Where 1) the first inequality is the original bound at θ'' (noting that H(q) does not change because it doesn't depend on θ), 2) the second inequality is the result of maximization, and 3) the equality is

the tightness at $\theta = \theta'$. In words, this means that each round of maximization which gives a new θ'' given some previous θ' can never decrease the log likelihood function.

So in conclusion, we use the expectation of the complete data log likelihood function because we can make it separable and maximizing it never decreases the log likelihood of the data given the conditional independence assumption. The first reason is due to the result in the EM theory section which shows the separability and thus the ease of maximization. The second reason is due to the result in this section. Namely that we can 1) rewrite the log likelihood function with an expectation trick, 2) use Jensen's inequality to get a lower bound on the log likelihood and decompose the lower bound family to show a familiar form, 3) note that one of the choices within the family of lower bounds makes it the expectation of the complete data log likelihood and entropy (constant as theta changes), 4) show that this choice of lower bound makes the inequality tight with KL divergence, and 5) make the final conclusion that maximizing Q over theta will never decrease the desired log likelihood function.

References

Dempster, A. P., Laird, N. M. and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B* 39(1), 1–38. doi:10.1111/j.2517-6161.1977.tb01600.x

Enamorado, T., Fifield, B. and Imai, K. (2019). Using a probabilistic model to assist merging of large-scale administrative records. *American Political Science Review* 113(2), 353–371. doi:10.1017/S0003055418000783

Fellegi, I. P. and Sunter, A. B. (1969). A theory for record linkage. *Journal of the American Statistical Association* 64(328), 1183–1210. doi:10.1080/01621459.1969.10501049

Jaro, M. A. (1989). Advances in record-linkage methodology as applied to matching the 1985 Census of Tampa, Florida. *Journal of the American Statistical Association* 84(406), 414–420.

Linacre, R., Lindsay, S., Manassis, T., Slade, Z., Hepworth, T., Kennedy, R. and Bond, A. (2022). Splink: Free software for probabilistic record linkage at scale. *International Journal of Population Data Science* 7(3). doi:10.23889/ijpds.v7i3.1794

McVeigh, B. S. and Murray, J. S. (2017). Practical Bayesian inference for record linkage. arXiv:1710.10558

Valliant, R., Dever, J. A. and Kreuter, F. (2018). *Practical Tools for Designing and Weighting Survey Samples*, 2nd ed. Springer. doi:10.1007/978-3-319-93632-1

Xu, H., Li, X., Shen, C., Hui, S. L. and Grannis, S. (2019). Incorporating conditional dependence in latent class models for probabilistic record linkage: Does it matter? *The Annals of Applied Statistics* 13(3), 1753–1790. doi:10.1214/19-AOAS1256